Project 1 Report

For this project, we used Python to build DFAs that count the number of strings of length n in languages L and L2. The alphabet is {a, b, c, d}. For L, every substring of length 6 must contain all four letters. L2 has the same requirement, but the middle two characters also have to be aa.

dfa_L

In dfa_L.py, we made a DFA that keeps track of the last 5 characters read. This is enough because we only need to check substrings of length 6. Each state represents a buffer of up to 5 characters, which we encode as an integer using base 4.

When a new character is added, if the buffer has fewer than 5 characters, it just gets appended. Otherwise, we check if the 6 characters contain all four letters. We used a bitmask for this, where each letter has its own bit. If the mask is 1111, all four letters are present. Otherwise, the DFA goes to a fail state.

Pseudocode:

```
for each buffer state:
    for each character:
        if buffer length < 5:
            append character
        else:
            if all 4 letters are present:
                remove oldest character
                append new character
            else:
                go to fail state

#  all states except fail are accepting
```

dfa_L2

For L2, we used a second DFA in dfa_L2.py. Since the middle two characters have to be aa, we can build the string from the middle outwards to both ends.

Each state stores two states from the original DFA, one for each half of the string. Each transition adds one character to each half, giving 16 possible combinations.

The main issue with this approach is that checking the two halves separately doesn't check substrings that cross the middle. To fix this, we wrote the crossed() function, which checks the 6-character windows that cross the middle, including the fixed aa.

We also only create states when they are reached, instead of generating every possible pair of states.

Pseudocode:

start with two buffers containing 'a'

while there are states to process:

    for each pair of characters:

        update both DFA states

        if either half fails:

            go to fail state

        else if crossing the middle is invalid:

            go to fail state

        else:

            add the new state if it doesn't exist

            save the transition

all states except fail are accepting

Counting Strings

In counting.py, we used dynamic programming to count how many strings of length n are accepted by a DFA. We start by assigning 1 to accepting states and 0 to rejecting states. Then, for each additional character, we add up the counts from all possible transitions.

This lets us count strings without actually generating them. We also only store the counts for the current and previous iteration to save memory.

Pseudocode:

```
for each state:
    count[state] = 1 if accepting, else 0
repeat n times:
    for each state:
        new_count[state] = 0
        for each character:
            new_count[state] += count[next_state]
    count = new_count
return count[start_state]
#  count[state] means the number of strings of length i that lead from that state to an accepting state
```

Running the Program

In main.py, we made a menu where the user can choose to count strings in L or L2 and enter a length from 1 to 300.

For L, we run the counting algorithm with length n. For L2, odd lengths return 0 since there can't be two middle characters. For even lengths, we use n/2 - 1 transitions because each transition adds two characters and the middle aa is already included.